

23-813-0704: Computational Linguistics Lab

Lab Cycle 1 - Regular Expressions

Date: 16/07/2026

You are free to use any available public dataset for the questions that require training data. If you are curating your own dataset, it has to be submitted along with the code.

1. Write regular expressions for the following languages and implement them using Python library. By “word”, we mean an alphabetic string separated from other words by whitespace, any relevant punctuation, line breaks, and so forth.
 - a. The set of all strings with two consecutive repeated words (e.g., “Humbert Humbert” and “the the” but not “the bug” or “the big bug”);
 - b. all strings that start at the beginning of the line with an integer and that end at the end of the line with a word;
 - c. all strings that have both the word grotto and the word raven in them (but not, e.g., words like grottos that merely contain the word grotto);
 - d. Write a pattern that places the first word of an English sentence in a register. Deal with punctuation.

The program reads input from a file and prints output to the terminal.

2. Implement an ELIZA-like program, using substitutions. Choose a domain in which your program can legitimately engage in simple conversations. The program should exit when the user types “BYE BYE”
3. Implement a simple rule-based Text tokenizer for the English language using regular expressions. Your tokenizer should consider punctuation and special symbols as separate tokens. Contractions like “isn't” should be treated as 2 tokens: “is” and “n't”. Also identify abbreviations (eg, U.S.A) and internal hyphenation (eg., ice-cream) as single tokens.
4. Design and implement a Finite State Automata(FSA) that accepts English plural nouns ending with the character ‘y’, e.g. boys, toys, ponies, skies, and puppies but not boies or toies or ponys. (Hint: Words that end with a vowel followed by ‘y’ are appended with ‘s’ and will not be replaced with “ies” in their plural form).
5. Design and implement a Finite State Transducer(FST) that accepts lexical forms of English words(e.g. shown below) and generates its corresponding plurals, based on the e-insertion spelling rule $\epsilon \Rightarrow e / \{x,s,z\}^{\wedge} _ s^{\#}$
^ is the morpheme boundary and # - word boundary

Input	Output
fox^s#	foxes
boy^s#	boys

6. Implement a Byte Pair tokenizer for English using a small, representative corpus. Print each intermediate step in vocabulary creation. Discuss your choices and their impact.

Lab Cycle 2

Date: 28/07/26

7. Implement the *Minimum Edit Distance* algorithm to find the edit distance between any two given strings. Also, list the edit operations.
8. Design and implement a statistical spell checker for detecting and correcting non-word spelling errors in English, using the bigram language model. Your program should do the following:
 - a. Tokenize the corpus and create a vocabulary of unique words.
 - b. Create a bi-gram frequency table for all possible bigrams in the corpus.
 - c. Scan the given input text to identify the non-word spelling errors
 - d. Generate the candidate list using 1 edit distance from the misspelled words
 - e. Suggest the best candidate word by calculating the probability of the given sentence using the bigram LM.
9. Implement a noisy channel spell checker for non-word spelling errors in the English language, using the same dataset. Compare its performance with the bigram LM spellchecker from the previous question.
10. Design and implement a Binary Logistic Regression classifier to detect and correct real-word homophone errors in input text using the provided confusion sets:

Your system must support the following confusion sets: {"write", "right", "rite"}, {"peace", "piece"}, {"their", "there", "they're"}.

Your program should do the following:

 - a. Scan input tokens. If a token belongs to any supported confusion set, trigger candidate evaluation.
 - b. Extract features for all candidates in that specific confusion set.
 - c. Use your trained Logistic Regression model to predict $P(y=1 | \text{candidate, context})$ for each candidate.
 - d. Replace the target word with the highest-scoring candidate.

Note: Because different confusion sets rely on different linguistic signals, your features must generalize across all three sets. Design at least 3 distinct feature types (e.g., n-gram log-probabilities, unigram priors, syntax/POS indicators, exact-match flags).

11. Explore any available neural spelling error detection and correction model. How does the performance of your previous models compare with the neural model evaluated on your chosen dataset?
12. Write a program
 - a. To compute unsmoothed unigrams and bigrams from a small corpora of your choice (Use only the train set)
 - b. Add an option to compute the probability of any text entered by the user.
 - c. Add an option to generate random sentences using (i) Unigram, (ii) Bigrams. Compare the quality of your generated sentences.
 - d. Add an option to your program to compute the perplexity of a test set. Compare the perplexity of unigram and bigram models.
13. Implement a text classifier for sentiment analysis using the Naive Bayes theorem. Use Add-k smoothing to handle zero probabilities. Compare the performance of your classifier for k values 0.25, 0.75, and 1.

Lab Cycle 3

Date: 11/8/26

14. Implement the Viterbi algorithm to find the most probable POS tag sequence for a given sentence, using the given probabilities:

a_{ij}	STOP	NN	VB	JJ	RB
START	0	0.5	0.25	0.25	0
NN	0.25	0.25	0.5	0	0
VB	0.25	0.25	0	0.25	0.25
JJ	0	0.75	0	0.25	0
RB	0.5	0.25	0	0.25	0

b_{ik}	time	flies	fast
NN	0.1	0.01	0.01
VB	0.01	0.1	0.01
JJ	0	0	0.1
RB	0	0	0.1

15. Write a Python code to calculate bigrams from a given corpus and calculate the probability of any given sentence
16. Write a program to compute the TF-IDF matrix given a set of training documents. Also, calculate the cosine similarity between any two given documents or two given words.
17. Write a program to compute the PPMI matrix given a set of training documents. Also, calculate the cosine similarity between any two given documents or two given words.
18. Implement a Naive Bayes classifier with add-1 smoothing using the given test data and disambiguate any word in a given test sentence. Use Bag-of-words as the feature. You may define your vocabulary.

Sample Input :

No.	Sentence	Sense
1	I love fish. The smoked <u>bass</u> fish was delicious.	fish
2	The <u>bass</u> fish swam along the line.	fish
3	He hauled in a big catch of smoked <u>bass</u> fish.	fish
4	The <u>bass</u> guitar player played a smooth jazz line.	guitar

Test Sentence: He loves jazz. The bass_line provided the foundation for the guitar solo in the jazz piece

Test word: bass

Output: guitar

Lab Cycle 4

Date: 27/8/26